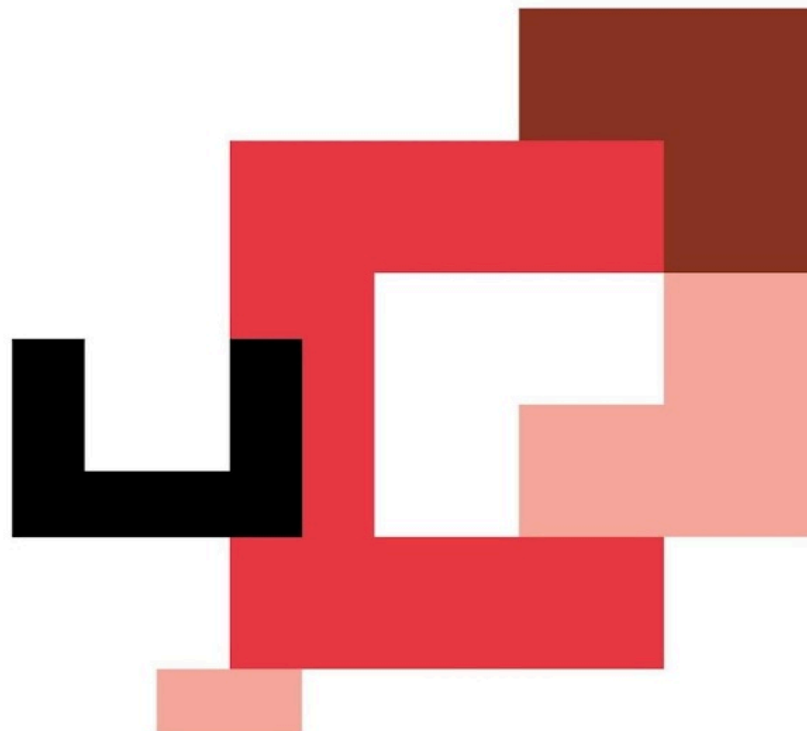




FULL STACK DEVELOPER + IA

Módulo 2 | Proyecto backend



Guía de trabajo — Semanas de Proyecto

Objetivo: Backend 100% React Ready

Durante estas **dos semanas de proyecto** el objetivo principal es **dejar el backend completamente preparado para ser consumido por un frontend en React**.

El proyecto que habéis construido durante el módulo ya tiene muchas funcionalidades:

- autenticación con cookies
- productos
- reviews
- wishlist
- carrito
- checkout
- deploy en producción
- documentación con Swagger

Ahora el objetivo es **consolidar y limpiar el proyecto** para que pueda ser utilizado sin problemas por un frontend.

La prioridad absoluta de estas semanas es:

None

Backend React Ready

Después, como mejoras adicionales:

1. Cloudinary (subida de imágenes)
2. Supertest (tests de endpoints)

Prioridad número 1 — Backend React Ready

El backend debe quedar **completamente preparado para que un frontend React pueda consumir la API sin problemas**.

Esto significa que todo debe estar:

- limpio

- organizado
 - probado
 - documentado
 - consistente
-

Estructura recomendada del proyecto

El proyecto debería tener una estructura clara similar a esta:

None

src

|

└ config

| └ database.js

|

└ controllers

| └ auth.controller.js

| └ product.controller.js

| └ cart.controller.js

| └ review.controller.js

| └ wishlist.controller.js

|

└ services

| └ auth.service.js

| └ product.service.js

| └ cart.service.js

| └ review.service.js

```
|   └─ wishlist.service.js
|
|─ routes
|   └─ auth.routes.js
|   └─ product.routes.js
|   └─ cart.routes.js
|   └─ review.routes.js
|   └─ wishlist.routes.js
|
|─ middlewares
|   └─ auth.middleware.js
|   └─ requireRole.js
|   └─ errorHandler.js
|   └─ notFound.js
|
|─ app.js
└─ server.js
```

Estructura mínima que debe tener el proyecto

Servidor

Debe existir separación entre:



None

`server.js`

`app.js`

Ejemplo conceptual:

None

`server.js` → arranca el servidor

`app.js` → configura middlewares y rutas

Configuración de base de datos

Debe existir conexión correcta con:

None

`PostgreSQL (Supabase)`

`MongoDB (Atlas)`

Uso esperado:

None

`SQL` → usuarios, productos, carrito, pedidos

`Mongo` → reviews, wishlist

Autenticación

El sistema de autenticación debe estar completamente funcional.

Debe incluir:

None

`registro`

login

JWT

middleware auth

roles de usuario

Endpoints esperados:

None

POST /api/auth/register

POST /api/auth/login

GET /api/me

Productos

El sistema de productos debe permitir:

None

crear producto

leer productos

actualizar producto

eliminar producto

Endpoints:

None

GET /api/products

GET /api/products/:id

POST /api/products

```
PUT /api/products/:id
```

```
DELETE /api/products/:id
```

Reviews (MongoDB)

El sistema de reviews debe funcionar correctamente.

Ejemplo:

None

```
GET /api/products/:id/reviews
```

```
POST /api/products/:id/reviews
```

Wishlist (MongoDB)

El usuario debe poder guardar productos en su lista de favoritos.

Endpoints:

None

```
GET /api/wishlist
```

```
POST /api/wishlist/:productId
```

Carrito

El sistema de carrito debe funcionar correctamente.

Endpoints esperados:

None

```
GET /api/cart
```

```
POST /api/cart/items
```

```
DELETE /api/cart/items/:itemId
```

Checkout

El flujo de compra debe estar completo.

Proceso:

None

```
usuario añade productos al carrito
```

```
usuario consulta carrito
```

```
usuario hace checkout
```

```
se crea un pedido
```

```
el carrito pasa a CHECKED_OUT
```

Endpoint principal:

None

```
POST /api/cart/checkout
```

Respuestas consistentes para React

El backend debe devolver respuestas consistentes para que React pueda consumirlas fácilmente.

Ejemplo correcto:

JSON

```
{
```



```
"success": true,  
"data": []  
}
```

Errores:

```
JSON  
{  
  "success": false,  
  "error": "Product not found"  
}
```

CORS configurado

El backend debe permitir peticiones desde un frontend.

Ejemplo:

```
None  
npm install cors
```

Configuración:

```
JavaScript  
app.use(cors())
```

Swagger funcionando

La documentación de la API debe estar disponible en:

None

`/api/docs`

Esto permitirá probar endpoints fácilmente.

Uso de variables de entorno

Las variables sensibles no deben estar en el código.

Ejemplo:

None

`DATABASE_URL`

`MONGO_URI`

`JWT_SECRET`

`PORT`

Deploy funcionando

El proyecto debe estar desplegado en producción.

Ejemplo:

None

`Render`

Cada vez que se haga:

None

`git push`

el proyecto debe desplegarse automáticamente.

Refactorización y limpieza del código

Antes de añadir nuevas funcionalidades es muy importante:

- limpiar código
 - eliminar código duplicado
 - mejorar nombres de funciones
 - revisar errores
 - mejorar estructura
-

Uso de IA durante el proyecto

Se puede utilizar IA para:

- revisar lógica
- mejorar código
- detectar errores
- optimizar estructura

Pero es importante:

None

entender siempre qué hace el código

La IA debe usarse como **copiloto**, no como sustituto.

Mejora opcional 1 — Cloudinary

Una vez que el backend esté **React Ready**, se puede implementar subida de imágenes con Cloudinary.

Esto permitirá subir imágenes de productos.

Flujo típico:



None

Frontend

↓

envía imagen

↓

Backend

↓

Cloudinary

↓

devuelve URL

↓

guardar URL en base de datos

En la base de datos solo se guarda:

None

imageUrl

Esto permite que React pueda mostrar las imágenes fácilmente.

Habrà un **manual específico** para implementarlo.

Mejora opcional 2 — Supertest

Supertest permite **testear endpoints automáticamente**.

Hasta ahora habéis probado la API con:

None

Postman

Swagger

Supertest permite hacerlo desde código.

Ejemplo:

JavaScript

```
import request from "supertest"

import app from "../app"

test("GET products", async () => {

  const res = await request(app).get("/api/products")

  expect(res.statusCode).toBe(200)

})
```

Esto permite comprobar que la API sigue funcionando después de hacer cambios.

También habrá un **manual específico para implementarlo**.

Orden de prioridad del proyecto

Durante estas dos semanas el trabajo debe seguir este orden:

Prioridad 1

None

Backend React Ready

Todo el backend funcionando perfectamente.

Prioridad 2

None

Cloudinary

Subida de imágenes para productos.

Prioridad 3

None

Supertest

Tests automáticos de endpoints.

Objetivo final del proyecto

Al finalizar estas semanas el backend debería tener:

None

API completa de e-commerce

deploy en producción

documentación Swagger

backend preparado para React

subida de imágenes

tests automáticos